

Optimisation des Systèmes de Liquidation Haute Fréquence sur Solana : Analyse des Latences et Architectures Prédictives

L'écosystème de la finance décentralisée sur Solana a atteint, en 2026, un stade de maturité technologique où la compétition pour l'extraction de valeur ne repose plus uniquement sur la logique algorithmique, mais sur une maîtrise absolue de la topologie du réseau et de la hiérarchie de l'information. L'analyse du bot de liquidation Jawas révèle des défaillances structurelles communes aux architectures de première génération, lesquelles se heurtent à la réalité d'un réseau dont le temps de bloc tend vers les 400 millisecondes et où la finalité des transactions est désormais optimisée par des protocoles comme Alpenglow.¹ Le problème exposé ne réside pas dans la puissance de calcul brute du bot, mais dans son positionnement au sein du pipeline de données de Solana. Pour un liquidateur, recevoir une information on-chain via un flux standard équivaut à consulter les archives d'un événement déjà consommé par des acteurs opérant plus près de la source de vérité, à savoir le leader du slot et les flux d'oracles hors-chaîne.²

Diagnostic de l'Architecture Réactive et Goulots d'Étranglement Structurels

Le fonctionnement actuel de l'outil Jawas, tel que décrit dans la note technique de mai 2026, repose sur une approche principalement réactive. Le bot écoute des signaux on-chain pour qualifier une cible après que l'événement déclencheur s'est produit. Cette dépendance à un signal tardif constitue le premier goulot d'étranglement majeur.⁴ Dans un marché efficient, la liquidation est le résultat final d'une chaîne de causalité qui commence souvent sur un échange centralisé (CEX) et se propage par les oracles avant de se manifester sur le carnet d'ordres ou le protocole de prêt.³

L'utilisation de WebSockets pour surveiller les programmes introduit une latence de propagation significative. Bien que les WebSockets soient préférables au polling HTTP, ils demeurent tributaires de la reconstruction des blocs par le fournisseur RPC et de la sérialisation JSON, un format lourd et coûteux en temps de traitement par rapport aux protocoles binaires.⁶ En 2026, un retard de 200 millisecondes n'est plus une simple marge d'erreur, mais une éternité technologique séparant le gagnant du slot de tous les autres participants.⁶

Analyse Comparative des Mécanismes de Réception de Données

Le tableau suivant détaille la hiérarchie des flux de données disponibles sur Solana et leur impact sur la latence de détection pour un moteur de liquidation.

Type de Flux	Position dans le Pipeline	Latence Moyenne (P90)	Nature de la Donnée
Standard RPC / WS	Post-propagation Turbine	> 200 ms	JSON-RPC (Texte) ⁶
Enhanced WebSockets	Agrégation gRPC vers JSON	150 - 200 ms	JSON-RPC (Optimisé) ⁷
Yellowstone gRPC	Sortie directe Geyser	5 - 15 ms	Protobuf (Binaire) ²
ShredStream	Flux de shreds leaders	< 5 ms	Shreds bruts (UDP) ⁶

La dépendance de Jawas aux lectures RPC supplémentaires pour enrichir les événements (qualification de la cible) ajoute une couche de latence fatale sur le chemin critique.⁴ Chaque appel `getAccountInfo` ou `getProgramAccounts` après la réception d'un signal initial consomme des millisecondes précieuses. Les liquidateurs d'élite maintiennent l'intégralité de l'état du protocole cible (shadow state) en mémoire vive, éliminant ainsi tout besoin de requête externe au moment de l'exécution.⁸

L'Absence de Simulation sur le Chemin Critique

Un point critique soulevé dans l'analyse de Jawas concerne l'absence de simulation complète sur le chemin de production ("Hot Path").⁴ Si cette omission vise à gagner du temps de calcul, elle se traduit par l'envoi de transactions vouées à l'échec car l'obligation peut être redevenue saine (healthy) entre le signal et l'inclusion.⁴ Les systèmes modernes intègrent des moteurs de simulation locale ultra-rapides capables de valider l'état d'une obligation en microsecondes, permettant de filtrer les opportunités périmées avant même qu'elles n'atteignent le réseau.²

La Hiérarchie de l'Information et l'Avantage des Oracles Pull

L'une des raisons principales pour lesquelles un liquidateur est battu systématiquement réside dans sa gestion des flux de prix. Les protocoles de prêt comme Kamino Finance s'appuient sur des oracles pour évaluer la santé des positions.⁸ En 2026, Pyth Network est devenu le standard

de fait avec son modèle "Pull Oracle" via Hermes.⁵

Dans l'ancien modèle "Push", l'oracle mettait à jour le prix on-chain à intervalles réguliers. Le liquidateur n'avait qu'à attendre cette mise à jour. Dans le modèle "Pull", le prix n'est pas nécessairement mis à jour sur la blockchain de manière autonome. C'est l'utilisateur de l'information (le liquidateur ou le protocole) qui doit injecter le prix signé le plus récent dans la transaction de liquidation.¹² Si un concurrent surveille les prix sur les CEX ou via le réseau Hermes de Pyth et voit un mouvement de prix avant qu'il ne soit reflété on-chain, il peut construire une transaction qui met à jour le prix et liquide la position dans le même bloc atomique.³

Mécanisme de l'Avantage de Latence de l'Oracle

Le liquidateur subissant des échecs de rapidité doit comprendre que la "vérité" du prix circule selon un cycle spécifique. Les fournisseurs de données institutionnels (market makers, bourses) publient des prix vers Pythnet avec une fréquence infra-seconde.⁵

1. **Origine** : Mouvement de prix sur un carnet d'ordres centralisé (ex: Binance/Coinbase).
2. **Capture** : Les éditeurs Pyth transmettent le nouveau prix à Pythnet.⁵
3. **Diffusion** : Le réseau Hermes rend le message signé disponible via API.¹²
4. **Exécution** : Le bot de tête récupère ce message et l'inclut dans son instruction de liquidation sur Solana.¹³

Le bot Jawas semble attendre que la liquidation soit "observable" on-chain, ce qui signifie qu'il intervient à l'étape 5 d'un processus où ses concurrents ont déjà agi à l'étape 4.⁴ L'implication est profonde : pour gagner, il ne faut pas surveiller la blockchain, mais surveiller les sources qui vont modifier la blockchain.³

Ingénierie de l'Exécution et Dynamiques du MEV sur Solana

Une fois l'opportunité détectée et la transaction construite, le défi se déplace vers l'inclusion et l'ordonnancement. Sur Solana, la notion de "Premier Arrivé, Premier Servi" est nuancée par les mécanismes de Maximal Extractable Value (MEV) et le système de frais de priorité.¹⁵

Jito-Solana et l'Économie des Bundles

En 2026, Jito-Solana est le client validateur dominant, traitant plus de 90% du stake du réseau.¹⁶ Il a introduit un marché d'enchères de blocs où les "chercheurs" (searchers) soumettent des bundles de transactions accompagnés d'un pourboire (tip).¹⁵ Les liquidations sont des transactions à haute valeur ajoutée qui entrent directement en compétition dans ces enchères.

Le bot Jawas, s'il utilise des frais de priorité standards sans passer par les bundles Jito, se retrouve dans la file d'attente générale (Gulf Stream) tandis que les concurrents utilisent des canaux privés vers les moteurs de blocs de Jito.¹⁵ Le pourboire Jito garantit non seulement

l'inclusion, mais souvent une position atomique spécifique dans le bloc, évitant ainsi d'être devancé par un arbitrage qui rendrait la liquidation invalide.¹⁵

Comparaison des Stratégies d'Inclusion

Stratégie	Mécanisme	Garantie d'Ordre	Coût Associé
Frais de Priorité Natifs	Enchère de Compute Units	Faible (dépend du leader)	Variable (SOL) ¹⁸
Jito Bundles	Enchère hors-chaîne atomique	Maximale	Pourboire (Tip) ¹⁵
Direct TPU Sends	Envoi UDP/QUIC au leader	Moyenne	Infrastructures réseaux ¹⁹
Warp / Relais Privés	Réseau de fibre dédié	Élevée	Abonnements Premium ¹¹

L'analyse de Jawas mentionne que la fenêtre utile est souvent "consommée par un autre".⁴ Cela suggère que même si la transaction arrive dans le bon slot, elle est classée après celle d'un concurrent ayant payé un pourboire plus élevé ou ayant utilisé un relais plus rapide pour atteindre le port TPU du leader.¹¹

Gestion de l'État Local et Algorithmique de Risque

Pour éliminer la latence de qualification, un bot doit posséder une connaissance parfaite de l'état du protocole de prêt. Kamino Finance utilise un modèle de santé complexe basé sur des pondérations d'actifs et des seuils de liquidation spécifiques.⁸

Modélisation du Facteur de Santé (Health Factor)

Le facteur de santé d'une obligation dans un protocole comme Kamino ou Solend peut être défini mathématiquement. Pour une position avec plusieurs collatéraux et dettes, la condition de liquidation est déclenchée par :

$$H = \frac{\sum_i (C_i \times P_i \times LTV_i)}{\sum_j (D_j \times P_j)} \leq 1.0$$

Où :

- C_i est la quantité de l'actif de collatéral i .

- P_i est le prix de l'actif i (ajusté par l'oracle).
- LTV_i est le "Liquidation Loan-To-Value" pour l'actif i .²⁰
- D_j et P_j représentent la quantité et le prix de la dette j .

Un bot performant ne calcule pas ce ratio de manière ponctuelle. Il maintient un "Shadow Ledger" en local, mis à jour par un flux gRPC Yellowstone.² À chaque fois qu'un prix P change sur une plateforme de trading ou qu'une mise à jour de compte gRPC modifie un C ou un D , le bot recalcule instantanément le vecteur de santé de toutes les positions affectées.⁸

Le "Hot State" et la Pré-construction de Transactions

La défaillance de Jawas réside dans le fait qu'il construit la transaction au moment de l'alerte.⁴ Une architecture de pointe pré-construit des transactions pour chaque obligation dont le facteur de santé est proche de 1.1 ou 1.05.²⁰ Ces transactions sont stockées en mémoire, déjà signées, avec des champs de prix laissés vides ou mis à jour par un fil d'exécution séparé. Lorsque le seuil de 1.0 est franchi, l'action de soumission est une simple opération d'E/S réseau, sans aucune logique de calcul sur le chemin critique.²

Infrastructure Physique et Topologie Réseau en 2026

La vitesse de la lumière et les sauts de routage réseau imposent des limites physiques incompressibles. En 2026, la distribution géographique des validateurs Solana est concentrée dans des régions spécifiques pour optimiser la latence de vote.²¹

Colocalisation et Régions AWS

La majorité du stake de Solana se trouve dans des centres de données en Europe (Francfort, Amsterdam) et dans certaines régions des États-Unis et d'Asie.¹¹ Un bot de liquidation opérant depuis une région AWS éloignée du leader actuel subit une pénalité de latence de plusieurs dizaines de millisecondes avant même que ses paquets n'atteignent le réseau Solana.¹¹

Région Stratégique	Avantage de Colocalisation	Part du Stake Estimée
AWS eu-central-1 (Francfort)	Proximité des clusters européens majeurs	~35% ²¹
AWS ap-southeast-1 (Singapour)	Hub principal pour les leaders asiatiques	~20% ²¹

Equinix / Bare Metal (Tokyo/Amsterdam)	Faible latence vers les nœuds Firedancer	~25% ⁹
---	---	-------------------

Le bot Jawas doit impérativement être déployé sur du matériel "Bare Metal" optimisé, capable de gérer des interruptions réseau à haute fréquence et d'utiliser des technologies comme DoubleZero pour contourner l'internet public et atteindre les validateurs via des dorsales en fibre optique privées.⁹

Firedancer et l'Optimisation du Matériel

L'introduction de Firedancer en 2025-2026 a relevé les exigences matérielles. Un validateur Firedancer peut traiter des millions de transactions par seconde, ce qui signifie que le goulot d'étranglement n'est plus le réseau, mais la capacité du bot à ingérer et traiter ces flux.⁹ Une configuration recommandée pour un bot de liquidation en 2026 inclut des processeurs à haute fréquence (3.8+ GHz), au moins 512 Go de RAM ECC DDR5 et des disques NVMe Gen5 pour le stockage temporaire des comptes.⁹

Évolutions du Réseau : Alpenglow, ACE et IBRL

Solana en 2026 n'est plus le même réseau qu'en 2023. Trois mises à jour majeures ont transformé les opportunités de liquidation : Alpenglow, ACE et IBRL.¹

Alpenglow et la Finalité Prévisible

Alpenglow est une refonte du consensus qui stabilise la production de blocs et réduit l'incertitude sur l'ordonnancement des transactions.¹ Pour un liquidateur, cela signifie que la finalité d'une transaction est atteinte en moins de 150 millisecondes.¹¹ Cela permet aux moteurs de liquidation de réallouer leur capital (repay) beaucoup plus rapidement, augmentant l'efficacité globale du bot sur une période de haute volatilité.¹

Application-Controlled Execution (ACE)

Le modèle ACE permet aux applications comme Kamino de définir des contraintes d'exécution spécifiques.¹¹ Cela peut inclure des mécanismes de protection des utilisateurs contre les liquidations prédatrices ou des fenêtres d'enchères spécifiques. Un bot doit être "ACE-aware", c'est-à-dire capable d'adapter ses transactions aux règles de priorité définies par le contrat intelligent lui-même, et non plus seulement par le réseau.¹¹

IBRL : Bandwidth and Latency Improvements

Le programme IBRL vise à aligner les performances de Solana sur celles des réseaux financiers traditionnels (FX, dérivés).¹ Pour le développeur de Jawas, cela signifie que la compétition ne va faire que s'intensifier. La "fenêtre utile" mentionnée dans le rapport initial va continuer à rétrécir, rendant l'investissement dans des flux de données à ultra-basse latence (ShredStream) non

plus optionnel, mais vital pour la survie du projet.⁶

Analyse du "Shadow State" et Recalcul Infinitésimal

Le cœur technique de la supériorité des liquidateurs concurrents réside dans leur capacité à simuler l'état futur du réseau avant qu'il ne se produise. C'est ce qu'on appelle le shadowing d'état local.⁸

Lorsqu'un bot reçoit un flux gRPC Yellowstone, il ne se contente pas de lire les messages. Il possède une instance locale de la machine virtuelle Solana (SVM) ou un moteur de calcul simplifié qui applique chaque changement de solde ou de prix à son propre registre en mémoire.²

La Problématique de l'Enrichissement des Données

Le rapport Jawas indique que le bot doit effectuer des "lectures RPC" supplémentaires pour qualifier une cible.⁴ C'est ici que se perd la course. Un bot configuré correctement possède déjà les informations suivantes en cache local :

- Les soldes exacts de chaque obligation.
- Les adresses des comptes de tokens associés (ATA) nécessaires pour le remboursement.
- Les paramètres de risque actuels du pool (LTV, bonus de liquidation).
- Les derniers messages signés des oracles Pyth pour tous les actifs concernés.³

Le processus de décision doit passer d'un modèle séquentiel à un modèle parallèle. Pendant que le fil d'exécution A surveille les prix hors-chaîne, le fil B pré-calcule les deltas de santé, et le fil C maintient une connexion persistante avec le moteur de bundles Jito pour minimiser le temps de handshake au moment de l'envoi.²

Synthèse des Causes de Retard et Recommandations

L'échec systématique de Jawas peut être attribué à une cascade de délais, chacun paraissant mineur isolément, mais dont le cumul excède la fenêtre d'opportunité dictée par les concurrents les plus performants.

1. **Latence de Détection (100-200 ms) :** Utilisation de WebSockets JSON-RPC au lieu de flux binaires gRPC ou ShredStream.⁶
2. **Latence de Qualification (50-100 ms) :** Requêtes RPC supplémentaires au lieu d'un état local complet.⁴
3. **Latence de Construction (20-50 ms) :** Rafraîchissement des réserves et création d'ATA au moment de l'action au lieu d'un pré-calcul.⁴
4. **Latence d'Inclusion (400 ms+) :** Absence de pourboires Jito ou de bundles, reléguant la transaction aux files d'attente publiques saturées.¹⁵
5. **Latence d'Anticipation (Infinie) :** Attente de la mise à jour on-chain des prix au lieu d'utiliser le modèle pull de Pyth pour forcer la mise à jour.⁵

Plan d'Action pour la Refonte Technique

La transformation du bot doit suivre une trajectoire de modernisation centrée sur la réduction drastique de l'entropie informationnelle.

Phase 1 : Migration de l'Infrastructure de Données

Le remplacement immédiat de l'Observer WebSocket par un client Yellowstone gRPC est la priorité absolue.² Ce client doit être configuré pour recevoir uniquement les mises à jour des comptes appartenant au programme Kamino Lend V2, réduisant ainsi le bruit de fond et la charge CPU.⁸ L'utilisation de Protobuf permettra une désérialisation beaucoup plus rapide que le parsing JSON.²

Phase 2 : Implémentation du Shadow State et Pré-calcul

Il est nécessaire de construire une base de données en mémoire (type Redis ou structure Rust native) qui stocke l'état complet de toutes les obligations actives.²⁰ Cette base doit être mise à jour en temps réel par le flux gRPC. Un algorithme de tri doit maintenir en permanence une liste des "Top 100" obligations les plus proches de l'insolvabilité.

Phase 3 : Intégration Jito et Orchestration des Bundles

Le bot doit s'intégrer au moteur de blocs Jito. Pour chaque liquidation potentielle, le bot doit préparer un bundle contenant :

1. L'instruction de mise à jour du prix Pyth (Hermes).¹²
2. L'instruction de liquidation Kamino.⁸
3. L'instruction de transfert de pourboire Jito.¹⁵

L'utilisation de pourboires dynamiques, ajustés en fonction de la valeur de la liquidation (bonus de 5-20% offert par les protocoles), permettra de rester compétitif sans sacrifier la rentabilité.²³

Phase 4 : Optimisation du Chemin de Soumission et Colocalisation

Le déploiement final doit être effectué dans la région AWS eu-central-1 (Francfort) sur une instance optimisée pour le calcul intensif (ex: c7g.metal).⁹ La connexion aux nœuds RPC et aux moteurs Jito doit se faire via des tunnels privés ou des connexions directes pour éliminer les délais de routage de l'internet public.¹¹

Conclusion sur la Viabilité des Stratégies de Liquidation

En conclusion, le bot Jawas ne souffre pas d'un défaut de logique, mais d'un manque de proximité avec les flux de données primaires de Solana. En 2026, la liquidation n'est plus une simple fonction de surveillance, c'est une opération d'ingénierie système qui nécessite une

intégration verticale complète, de la réception des shreds au positionnement atomique dans les bundles Jito.⁶ Le passage d'une architecture réactive à une architecture prédictive, combiné à une infrastructure colocalisée et à l'utilisation intelligente des oracles pull, est le seul chemin permettant de capturer la valeur dans un environnement de plus en plus disputé et technologiquement exigeant. Le succès futur du projet dépendra de sa capacité à "voir le futur" par l'anticipation des mouvements de prix et de l'état du réseau, plutôt que de réagir à un passé déjà consommé par la concurrence.³

Sources des citations

1. Solana in 2026: Technical Roadmap - Blockdaemon, consulté le mai 11, 2026, <https://www.blockdaemon.com/blog/solana-in-2026-technical-roadmap>
2. Complete 2026 Guide to Solana Streaming and Yellowstone gRPC, consulté le mai 11, 2026, <https://blog.triton.one/complete-guide-to-solana-streaming-and-yellowstone-grpc/>
3. Best Practices | Pyth Developer Hub, consulté le mai 11, 2026, <https://docs.pyth.network/price-feeds/core/best-practices>
4. `jasas_production_runtime.md`
5. Pyth Network Oracle: Real-Time Crypto Price Data Guide 2026 - Bitget, consulté le mai 11, 2026, <https://www.bitget.com/academy/pyth-network-oracle>
6. ShredStream vs Geyser vs Standard RPC: Choosing the Right Data ..., consulté le mai 11, 2026, <https://rpcfast.com/blog/shredstream-vs-geyser-vs-standard-rpc>
7. Introducing: Next-Generation Enhanced WebSockets - Helius, consulté le mai 11, 2026, <https://www.helius.dev/blog/introducing-next-generation-enhanced-websockets>
8. Solana DeFi Deep Dives: Kamino Automated Lending & Liquidity ..., consulté le mai 11, 2026, <https://medium.com/@Scoper/solana-defi-deep-dives-kamino-late-2025-080f6f52fa29>
9. Solana Validator Hardware vs Cloud 2026 Node Setup - Everstake, consulté le mai 11, 2026, <https://everstake.one/resources/blog/solana-validator-node-hardware-vs-cloud-2026>
10. 1480 people got liquidated on Kamino last month. Most had SOL collateral. - Reddit, consulté le mai 11, 2026, https://www.reddit.com/r/solana/comments/1pdak2q/1480_people_got_liquidated_on_kamino_last_month/
11. Solana trading infrastructure 2026: MEV, nodes, latency ..., consulté le mai 11, 2026, <https://chainstack.com/solana-trading-infrastructure-2026/>
12. Why Update Prices | Pyth Developer Hub, consulté le mai 11, 2026, <https://docs.pyth.network/price-feeds/core/why-update-prices>
13. What is Pyth Network | Oracle Explained | Gate Learn, consulté le mai 11, 2026, <https://www.gate.com/learn/articles/what-is-pyth-network/976>
14. Pyth Network - Sei Docs, consulté le mai 11, 2026,

- <https://docs.sei.io/evm/oracles/pyth-network>
15. Solana MEV Economics: Jito, Bundles, and Liquid Staking - Quicknode Blog, consulté le mai 11, 2026,
<https://blog.quicknode.com/solana-mev-economics-jito-bundles-liquid-staking-guide/>
 16. What Is Jito TipRouter? | Jito Foundation - Jito Network, consulté le mai 11, 2026,
<https://www.jito.network/blog/what-is-jito-tiprouter/>
 17. Jito's Role in Solana Deep Dive: MEV Tips and Transaction Prioritization Analysis | Gate Learn, consulté le mai 11, 2026,
<https://www.gate.com/learn/articles/jito-s-role-in-solana-deep-dive/8724>
 18. Production Readiness - Solana, consulté le mai 11, 2026,
<https://solana.com/docs/payments/production-readiness>
 19. TpuClient in solana_client::tpu_client - Rust - Docs.rs, consulté le mai 11, 2026,
https://docs.rs/solana-client/latest/solana_client/tpu_client/struct.TpuClient.html
 20. Solana Kamino Lend position monitor with liquidation risk analysis and Telegram alerts - GitHub, consulté le mai 11, 2026,
<https://github.com/csacanam/kamino-positions-monitor>
 21. Solana Validator Performance Report - Coinbase, consulté le mai 11, 2026,
<https://www.coinbase.com/de/blog/solana-validator-performance-report>
 22. klend-sdk/README_KAMINO_MANAGER.md at master - GitHub, consulté le mai 11, 2026,
https://github.com/Kamino-Finance/klend-sdk/blob/master/README_KAMINO_MANAGER.md
 23. GitHub - solendprotocol/liquidator: open source version of a liquidation bot running against solend, consulté le mai 11, 2026,
<https://github.com/solendprotocol/liquidator>